

Veermata Jijabai Technological Institute (VJTI)

Mumbai – 400 019

System Design Document

Part B: Object Design Refinement

Food Donation & Lifecycle Management System
(FDLMS)

Software Engineering Laboratory – Experiment 8B
Department of Computer Engineering

Student Name	Roll Number	Group
Rudrakshi Chincholkar	241071014	C
Nihira Neralwar	241071049	C

Academic Year: 2025–26 | April 2026

Table of Contents

Section	Title
1	Design Patterns Applied
1.1	Singleton – NotificationService Session Manager
1.2	State – FoodDonation Lifecycle FSM
1.3	Strategy – DonationMatchingEngine
1.4	Observer – Event-Driven Notification
1.5	Repository – Data Access Abstraction
1.6	Template Method – Report Generation
2	Missing Attributes and Operations
-	Table 2: Missing Attributes and Operations — Identified and Resolved
3	Type Signatures and Visibility
3.1	Enumerations

3.2	Class: User (Abstract)
3.3	Class: FoodDonation
3.4	Class: InspectionReport
3.5	Class: PickupTask
3.6	Class: DeliveryTask
3.7	Class: NGO
3.8	Class: NotificationService (Singleton)
4	Preconditions and Postconditions
4.1	FoodDonation.transition(newState, actor, notes)
4.2	InspectionReport.flagUnsafe(reason, hazard)
4.3	PickupTask.accept(volunteer)
4.4	DeliveryTask.confirmDelivery(proof, gps)
4.5	NotificationService.dispatch(userId, event, channels)

4.6	User.authenticate(pwd)
4.7	InspectionReport.attachPhoto(url)
5	Class and Method Invariants
-	References

1 Design Patterns Applied

The following Gang-of-Four (GoF) and enterprise design patterns have been identified and systematically applied to the FDLMS object model, derived from a review of use-case realisations produced in Experiments 4A and 4B.

1.1 Singleton – NotificationService Session Manager

Pattern: Singleton | **Category:** Creational (GoF) | **Applied to:** NotificationService

Rationale. The NotificationService is responsible for dispatching real-time alerts to all stakeholders (Donors, NGOs, Volunteers, Food Safety Inspectors, and Admins) across WebSocket, email, and SMS channels. A single, globally shared instance is mandatory to enforce the system-wide 3-second SLA (DG-N1), maintain a consistent retry queue (max 3 retries before dead-letter escalation), and prevent duplicate notification entries in the notifications table. Multiple independent instances would produce race conditions on the retry queue and violate INV-9 (exactly one database entry per dispatch call). A Singleton eliminates these failure modes while providing a centralised observability point for notification metrics on the Admin KPI dashboard.

Implementation.

- The constructor of NotificationService is declared private.
- A private static field instance : NotificationService holds the sole object.
- The static factory method getInstance() : NotificationService creates the instance on first call (lazy initialisation with double-checked locking for thread safety) and returns it on every subsequent call.
- All subsystem controllers (DonationController, InspectionController, PickupController, DeliveryController) call NotificationService.getInstance() instead of constructing their own dispatch objects.
- Modified class: NotificationService — +getInstance():NotificationService static method added; -NotificationService() constructor made private; -instance:NotificationService static field added; -retryQueue:List<NotifFailure> held as instance field.

1.2 State – FoodDonation Lifecycle FSM

Pattern: State | **Category:** Behavioural (GoF) | **Applied to:** FoodDonation + new DonationState hierarchy

Rationale. The food donation lifecycle traverses eight distinct states: SUBMITTED → INSPECTED → ACCEPTED → PICKUP_ASSIGNED → PICKED_UP → DELIVERED → RECONCILED, with REJECTED and CANCELLED as terminal exit states reachable from several intermediate stages. Each state imposes different transition guards, RBAC permissions, and downstream side-effects. Without the State pattern, FoodDonation.transition() would require deeply nested conditional chains that violate the Open/Closed Principle and make adding new lifecycle states (e.g., a future QUARANTINE state) error-prone. The State pattern encapsulates each state's allowed-transition table in a dedicated class, satisfying Design Goal DG-SU2 (immutable lifecycle logging) and eliminating branch-coverage gaps in unit tests.

Implementation.

- Interface DonationState declares handle(donation:FoodDonation, newState:DonationStatus):void.
- Concrete state classes: SubmittedState, InspectedState, AcceptedState, PickupAssignedState, PickedUpState, DeliveredState, ReconciledState, RejectedState, CancelledState.

- FoodDonation holds -currentState: DonationState and delegates transition() to currentState.handle().
- Each concrete state throws InvalidTransitionException for disallowed moves, keeping the guard logic co-located with the state rather than scattered across the aggregate.

Table: Garment Lifecycle FSM — Allowed Transitions

Current State	Allowed Next State(s)	Trigger Actor
SUBMITTED	INSPECTED, CANCELLED, REJECTED	Food Safety Inspector / Admin
INSPECTED	ACCEPTED, REJECTED	NGO / Admin
ACCEPTED	PICKUP_ASSIGNED, CANCELLED	Volunteer / Admin
PICKUP_ASSIGNED	PICKED_UP, CANCELLED	Volunteer
PICKED_UP	DELIVERED	Volunteer / Delivery Partner
DELIVERED	RECONCILED	System (auto)
RECONCILED	(terminal) —	—
REJECTED	(terminal) —	—
CANCELLED	(terminal) —	—

1.3 Strategy – DonationMatchingEngine

Pattern: Strategy | **Category:** Behavioural (GoF) | **Applied to:** DonationMatchingEngine

Rationale. The NGO-to-donation matching algorithm is a domain policy, not an invariant — it may need to change at runtime based on operational context (e.g., switching to expiry-urgency-first during a food glut, or proximity-first when logistics are constrained). Hard-coding one algorithm in DonationMatchingEngine would couple business logic to an implementation detail and require redeployment for every policy change. The Strategy pattern allows the Administrator to swap the algorithm through a configuration endpoint without modifying the engine, satisfying Design Goal DG-M2 (extensibility without altering existing interfaces).

Implementation.

- Interface MatchingStrategy declares match(donations:List<FoodDonation>, ngo:NGO):List<FoodDonation>.
- Concrete strategies: ProximityMatchingStrategy, ExpiryFirstMatchingStrategy, NGOPreferenceMatchingStrategy, BalancedWeightedStrategy.
- DonationMatchingEngine holds -strategy:MatchingStrategy and exposes +setStrategy(s:MatchingStrategy):void (Admin-only endpoint).
- A default of ProximityMatchingStrategy is set in the constructor so the field is never null (see INV-15).

1.4 Observer – Event-Driven Notification

Pattern: Observer | **Category:** Behavioural (GoF) | **Applied to:** Notification dispatch across FoodDonation, InspectionReport, PickupTask, DeliveryTask

Rationale. Multiple stakeholders — NGOs, Donors, Volunteers, Food Safety Inspectors, and Admins — require asynchronous notifications whenever lifecycle events occur (e.g., donation submitted, inspection flagged unsafe, pickup assigned). Directly invoking notification logic from entity classes couples domain objects to infrastructure channels, violates the Single Responsibility Principle, and makes test isolation impossible. The Observer pattern decouples event producers (domain transitions) from consumers (notification channels), satisfying DG-N1 (3-second SLA) and DG-N2 (configurable per-user notification preferences).

Implementation.

- Interface `DonationObserver` declares `update(event:DonationEvent):void`.
- `NotificationService` implements the observable registry:
`subscribe(observer:DonationObserver):void` and
`notifyObservers(event:DonationEvent):void`.
- Concrete observers: `EmailNotifier`, `WebSocketPushNotifier`, `SMSNotifier`.
- After any successful lifecycle transition, `FoodDonation.transition()` fires `notifyObservers(event)` and `NotificationService` fans out to all registered observers via an `@Async` thread pool, preventing lifecycle operations from blocking on I/O.

1.5 Repository – Data Access Abstraction

Pattern: Repository | **Category:** Enterprise (Domain-Driven Design) | **Applied to:** All persistence-bearing entity classes

Rationale. Business logic classes must not contain raw SQL or JPA criteria queries; this couples domain logic to database technology, prevents unit testing with mock data sources, and creates a maintenance burden when the schema evolves. The Repository pattern provides a collection-like interface over each aggregate root, satisfying DG-M1 (loose coupling) and DG-M3 (≥70% unit-test coverage for service layer).

Implementation.

- Generic interface: `Repository<T, ID>` with `findById(id:ID):Optional<T>`, `save(entity:T):T`, `delete(id:ID):void`, `findAll(spec:Specification<T>):List<T>`.
- Concrete repositories (Spring Data JPA): `DonationRepository`, `UserRepository`, `InspectionReportRepository`, `PickupTaskRepository`, `DeliveryTaskRepository`, `NotificationRepository`, `NGORepository`, `VolunteerRepository`, `LifecycleLogRepository`.

1.6 Template Method – Report Generation

Pattern: Template Method | **Category:** Behavioural (GoF) | **Applied to:** `KPIDashboardController` report generation pipeline

Rationale. Different report types — donation lifecycle report, NGO performance report, waste reduction report, volunteer activity report — share the same generation pipeline (fetch data → filter/aggregate → format → export) but differ only in the specifics of each step. Duplicating the pipeline in each report controller violates DRY and makes pipeline-level changes (e.g., adding an audit watermark) require editing every class. The Template Method pattern defines the invariant pipeline in one abstract class and delegates the variant steps to subclasses.

Implementation.

- Abstract class `ReportGenerator` declares
`+generate(params:ReportParams):ReportOutput` (template method) calling
`#fetchData()`, `#aggregate()`, `#format()`, `#export()`.
- Concrete subclasses: `LifecycleReportGenerator`, `NGOPerformanceReportGenerator`, `WasteReductionReportGenerator`, `VolunteerActivityReportGenerator`.

2 Missing Attributes and Operations

A systematic review of all use cases from Experiments 4A and 4B, cross-referenced against the FDLMS SRS (Experiment 3), revealed the following gaps in the initial object model produced in Experiment 8A. All gaps are now resolved in the refined model presented herein.

Table 2: Missing Attributes and Operations — Identified and Resolved

Class	Missing Member	Justification
User	+lastLoginAt: DateTime	Required for security audit log (UC-27) and session expiry enforcement.
User	+failedLoginCount: Int	Account lockout after 5 consecutive failures (boundary condition; INV-U4).
User	+isDeleted: Boolean	Soft-delete flag; data retained for audit trail compliance (DG-R2).
FoodDonation	+cancellationReason: String?	UC-04 requires the cancellation reason to be recorded for reporting and NGO transparency.
FoodDonation	+version: Int	Optimistic locking; prevents concurrent NGO acceptance race condition on the same donation.
FoodDonation	+ngoRef: NGO? (nullable)	Tracks which NGO accepted the donation; nullable until ACCEPTED state is reached.
FoodDonation	+expiryDate: LocalDate	Required by FSM guard (INV-4); a donation with already-expired food may not be submitted.
InspectionReport	+hazardLevel: HazardLevel	UI displays hazard level on Food Safety Inspector screens; required for UC-12 flagging.
InspectionReport	+assessedAt: DateTime?	Tracks when the inspector completed assessment; required for SLA reporting.
PickupTask	+acceptedAt: DateTime?	Required for acceptance time-window enforcement (UC-18); null until ASSIGNED.
PickupTask	+version: Int	Optimistic locking; prevents two volunteers from accepting the same task simultaneously.
DeliveryTask	+gpsConfirmation: Location?	UC-23 requires GPS co-ordinates as proof of physical pickup location.
DeliveryTask	+delayReason: String?	UC-25 (update delivery status) mandates storage of delay reason for NGO notification.
DeliveryTask	+proofOfDelivery: URL?	Delivery evidence (photo or signature URL); null until delivery is confirmed.

Volunteer	+availabilitySchedule: List<TimeSlot>	UC-19 (Set Availability) must persist a structured schedule for matching-engine filtering.
NGO	+serviceArea: List<Location>	Required by DonationMatchingEngine for proximity-first filtering of eligible donations.
NGO	+verificationStatus: VerificationStatus	Admin must verify NGO before it can accept donations (UC-07 onboarding flow).
FoodSafetyInspector	+assignedDonations: List<FoodDonation>	Tracks inspector workload; prevents over-assignment beyond configurable capacity limit.
NotificationService	+deliveryFailureLog: List<NotiffFailure>	UC-30 requires failed alerts to be logged and retried (max 3 times) before dead-letter escalation.
DonationMatchingEngine	+lastMatchTimestamp: DateTime	Cache invalidation: determines when last matching run was computed (DG-P3 staleness check).
LifecycleLog	All fields final (constructor-set)	Immutability invariant INV-14: INSERT-only; no UPDATE or DELETE permitted at DB level.
Donor	+totalDonationCount: Int	Aggregated KPI for donor dashboard and NGO trust-scoring; cached server-side.

3 Type Signatures and Visibility

This section provides the complete type-annotated, visibility-specified specification for all principal classes in the refined FDLMS object model. Visibility notation: + public, # protected, – private.

3.1 Enumerations

Table 3: Enumeration Types — FDLMS Refined Model

Enumeration	Values
UserRole	DONOR, NGO_REPRESENTATIVE, VOLUNTEER, DELIVERY_PARTNER, FOOD_SAFETY_INSPECTOR, SYSTEM_ADMIN
DonationStatus	SUBMITTED, INSPECTED, ACCEPTED, REJECTED, PICKUP_ASSIGNED, PICKED_UP, DELIVERED, RECONCILED, CANCELLED
InspectionOutcome	SAFE, UNSAFE, REQUIRES_REVIEW
HazardLevel	LOW, MEDIUM, CRITICAL
TaskStatus	PENDING, ASSIGNED, EN_ROUTE, ARRIVED, PICKED_UP, ISSUE_REPORTED, COMPLETED, CANCELLED
DeliveryStatus	ACCEPTED, IN_TRANSIT, DELAYED, DELIVERED
PenaltySeverity	WARNING, MINOR, MAJOR, BAN
VerificationStatus	PENDING_VERIFICATION, VERIFIED, SUSPENDED
ExportFormat	PDF, CSV, JSON, XLSX
NotificationChannel	WEBSOCKET, EMAIL, SMS

3.2 Class: User (Abstract)

Table 4a: Class: User (Abstract) — Attributes

V i s	Name	Type	Description
–	userId	UUID	Primary key (auto-generated)
–	passwordHash	String	SHA-256 salted hash; plain text never stored
#	email	String	Contact email (unique constraint)
#	role	UserRole	Immutable after creation; only Admin.changeRole() may update
+	username	String	Unique login identifier
+	isActive	Boolean	Account active flag
+	isDeleted	Boolean	Soft-delete flag; data retained for audit

+	createdAt	DateTime	Registration timestamp
+	lastLoginAt	DateTime?	Timestamp of last successful login; null before first login
–	failedLoginCount	Int	Consecutive failed login counter; resets on success

Table 4b: Class: User (Abstract) — Operations

Method Signature	Visibility	Return Type	Throws
authenticate(pwd:String):Boolean	public	Boolean	—
getRole():UserRole	public	UserRole	—
deactivate():void	public	void	IllegalStateException if already inactive
incrementFailedLogin():void	protected	void	AccountLockedException if count reaches 5
resetFailedLogin():void	public	void	—

3.3 Class: FoodDonation

Table 5a: Class: FoodDonation — Attributes

V i s	Name	Type	Description
–	donationId	UUID	Primary key
–	currentState	DonationState	State-pattern reference; delegates transition logic
–	status	DonationStatus	Current lifecycle enum; always consistent with lifecycleLog.last()
–	version	Int	Optimistic locking; prevents concurrent acceptance race
+	foodDescription	String	Human-readable description of donated food items
+	quantity	Double	Quantity in kg; must be > 0 (INV-5)
+	expiryDate	LocalDate	Expiry date; must be after submittedAt.toDate() (INV-4)
+	donorRef	Donor	FK → users; non-null at all times
+	ngoRef	NGO?	FK → ngo; null until ACCEPTED state
+	submittedAt	DateTime?	Set by submit(); null before submission
+	cancellationReason	String?	Set by cancel(); null if not cancelled

+	lifecycleLog	List<LifecycleEntry>	INSERT-only audit log; unmodifiable externally
---	--------------	----------------------	--

Table 5b: Class: FoodDonation — Operations

Method Signature	Visibility	Return Type	Throws
submit():void	public	void	IllegalStateException if status != null
transition(newState:DonationStatus, actor:User, notes:String):void	protected	void	InvalidTransitionException
cancel(reason:String, actor:Donor):void	public	void	IllegalStateException if status != SUBMITTED
getCurrentStatus():DonationStatus	public	DonationStatus	—
getLifecycleLog():List<LifecycleEntry>	public	List<LifecycleEntry>	—
assignNGO(ngo:NGO):void	protected	void	IllegalStateException if status != ACCEPTED

3.4 Class: InspectionReport

Table 6a: Class: InspectionReport — Attributes

V i s	Name	Type	Description
–	reportId	UUID	Primary key
+	outcome	InspectionOutcome?	Null until inspection completed
+	hazardLevel	HazardLevel?	Non-null iff outcome == UNSAFE (INV-8)
+	violationType	String?	Violation description; non-empty iff outcome == UNSAFE
+	inspectedAt	DateTime?	Non-null iff outcome is not null (INV-7)
+	assessedAt	DateTime?	Set when Admin reclassifies hazard level
+	photoEvidence	List<URL>	Uploaded evidence photos (max 5)
+	donationRef	FoodDonation	FK; 1:1 relationship (INV-6)
+	inspectorRef	FoodSafetyInspector	Assigned inspector FK

Table 6b: Class: InspectionReport — Operations

Method Signature	Visibility	Return Type	Throws
isSafe():Boolean	public	Boolean	—
flagUnsafe(reason:String, hazard:HazardLevel):void	public	void	IllegalArgumentException if reason blank; SecurityException if wrong inspector
generateReport(format:ExportFormat):BinaryData	public	BinaryData	IOException
attachPhoto(url:URL):void	public	void	IllegalArgumentException if url null; IllegalStateException if > 5 photos

3.5 Class: PickupTask

Table 7a: Class: PickupTask — Attributes

V i s	Name	Type	Description
–	taskId	UUID	Primary key
–	version	Int	Optimistic locking
+	status	TaskStatus	Current task status
+	donationRef	FoodDonation	FK
+	volunteerRef	Volunteer?	Null until ASSIGNED
+	assignedAt	DateTime?	Assignment timestamp
+	acceptedAt	DateTime?	Volunteer acceptance timestamp; null until accepted
+	scheduledPickupTime	DateTime	Planned pickup window

Table 7b: Class: PickupTask — Operations

Method Signature	Visibility	Return Type	Throws
accept(volunteer:Volunteer):void	public	void	ConcurrentModificationException; IllegalStateException if unavailable
markPickedUp(volunteer:Volunteer):void	public	void	SecurityException if wrong volunteer
cancel(reason:String):void	public	void	IllegalStateException if status == PICKED_UP

3.6 Class: DeliveryTask

Table 8a: Class: *DeliveryTask* — Attributes

V i s	Name	Type	Description
–	deliveryId	UUID	Primary key
+	status	DeliveryStatus	Current delivery status
+	donationRef	FoodDonation	FK
+	ngoRef	NGO	Receiving NGO FK
+	deliveryPartnerRef	DeliveryPartner?	Assigned partner; null until assigned
+	proofOfDelivery	URL?	Photo or signature URL; null until confirmed
+	deliveredAt	DateTime?	Null until delivery confirmed
+	gpsConfirmation	Location?	GPS co-ordinates at confirmation time
+	delayReason	String?	Populated if status transitions to DELAYED

Table 8b: Class: *DeliveryTask* — Operations

Method Signature	Visibility	Return Type	Throws
confirmDelivery(proof:URL, gps:Location):void	public	void	IllegalStateException if not IN_TRANSIT/DELAYED; IllegalArgumentException if proof null
reportDelay(reason:String):void	public	void	IllegalArgumentException if reason blank
assignPartner(p:DeliveryPartner):void	public	void	IllegalStateException if already assigned

3.7 Class: NGO

Table 9a: Class: *NGO* — Attributes

V i s	Name	Type	Description
–	ngoid	UUID	Primary key
+	organisationName	String	Legal name of the NGO
+	registrationNumber	String	Government registration identifier (unique)
+	serviceArea	List<Location>	Geographic service zones for proximity matching
+	preferredFoodTypes	List<String>	Matching engine preference filter

+	verificationStatus	VerificationStatus	Admin-managed; must be VERIFIED to accept donations
+	contactEmail	String	Primary contact for system notifications
+	isActive	Boolean	Soft operational status

Table 9b: Class: NGO — Operations

Method Signature	Visibility	Return Type	Throws
acceptDonation(donation:FoodDonation):void	public	void	IllegalStateException if verificationStatus != VERIFIED
getServiceArea():List<Location>	public	List<Location>	—
updatePreferences(types:List<String>):void	public	void	—

3.8 Class: NotificationService (Singleton)

Table 10a: Class: NotificationService (Singleton) — Attributes

V i s	Name	Type	Description
–	instance	NotificationService	Sole static instance (Singleton)
–	observerRegistry	List<DonationObserver>	Registered notification consumers
–	retryQueue	Queue<NotifFailure>	Failed notifications awaiting retry
+	deliveryFailureLog	List<NotifFailure>	Persistent log of all delivery failures

Table 10b: Class: NotificationService (Singleton) — Operations

Method Signature	Visibility	Return Type	Throws
getInstance():NotificationService	public static	NotificationService	—
dispatch(userId:UUID, event:DonationEvent, channels:List<NotificationChannel>):void	public	void	Never throws — failures logged and queued
subscribe(observer:DonationObserver):void	public	void	—
notifyObservers(event:DonationEvent):void	public	void	—

retryFailed():void	public	void	—
--------------------	--------	------	---

4 Preconditions and Postconditions

Method contracts are specified below using OCL-style precondition / postcondition / exception triples for all critical operations. Preconditions must hold before the method executes; postconditions describe the guaranteed system state after successful completion; exceptions describe the failure mode when a precondition is violated.

4.1 FoodDonation.transition(newState, actor, notes)

FoodDonation.transition(newState:DonationStatus, actor:User, notes:String):void	
Precondition	newState is a valid successor of this.status per the FSM allowed-transition table (Section 1.2). actor holds RBAC permission for this specific transition. this.status is not a terminal state (DELIVERED, RECONCILED, REJECTED, CANCELLED). this.isDeleted == false.
Postcondition	this.status == newState. A new LifecycleEntry with (fromState, toState, actor.userId, now(), notes) is appended to lifecycleLog; the entry is INSERT-only and immutable. All registered DonationObservers receive the transition event asynchronously within 3 seconds (DG-N1). The update is committed within a single ACID database transaction.
Exception	InvalidTransitionException if any precondition fails. this.status is unchanged; no LifecycleEntry is created; no notification is sent. The DB transaction is rolled back.

4.2 InspectionReport.flagUnsafe(reason, hazard)

InspectionReport.flagUnsafe(reason:String, hazard:HazardLevel):void	
Precondition	reason.length() > 0. hazard is a valid HazardLevel value. this.outcome != UNSAFE (cannot flag twice; prevents double-inspection). actor.role == FOOD_SAFETY_INSPECTOR and actor == this.inspectorRef (only the assigned inspector may flag).
Postcondition	this.outcome == UNSAFE. this.violationType == reason. this.hazardLevel == hazard. this.inspectedAt == now(). donation.transition(INSPECTED, actor, reason) is invoked. NGO and Donor receive real-time notification containing report summary within 3 seconds. Report is persisted atomically.
Exception	IllegalArgumentException if reason is blank. SecurityException if caller is not this.inspectorRef. IllegalStateException if outcome is already UNSAFE.

4.3 PickupTask.accept(volunteer)

PickupTask.accept(volunteer:Volunteer):void	
Precondition	this.status == PENDING (task not yet accepted by anyone). volunteer.isActive == true and volunteer.isDeleted == false. volunteer.availabilitySchedule overlaps with this.scheduledPickupTime window. Database-level optimistic lock acquired (SELECT FOR UPDATE on version field).

Postcondition	this.status == ASSIGNED. this.volunteerRef == volunteer. this.acceptedAt == now(). donation.transition(PICKUP_ASSIGNED, volunteer, "Volunteer accepted") is invoked. Donor and NGO notified. this.version incremented; optimistic lock released.
Exception	ConcurrentModificationException (optimistic lock violation) if another volunteer accepted simultaneously; this task is rolled back and volunteer must retry. IllegalStateException if volunteer is unavailable or task is already assigned.

4.4 DeliveryTask.confirmDelivery(proof, gps)

DeliveryTask.confirmDelivery(proof:URL, gps:Location):void	
Precondition	this.status == IN_TRANSIT or DELAYED. proof URL is non-null and syntactically valid. gps is non-null (required for UC-23 GPS verification). this.deliveredAt is null (delivery not already confirmed). recipient availability flag is set.
Postcondition	this.status == DELIVERED. this.deliveredAt == now(). this.proofOfDelivery == proof. this.gpsConfirmation == gps. donation.transition(DELIVERED, actor, "Delivery confirmed") is invoked. NGO and Donor notified. Reconciliation batch job enqueued for auto-RECONCILED transition.
Exception	IllegalStateException if food was not picked up or delivery already confirmed. IllegalArgumentException if proof or gps is null.

4.5 NotificationService.dispatch(userId, event, channels)

NotificationService.dispatch(userId:UUID, event:DonationEvent, channels:List<NotificationChannel>):void	
Precondition	userId is valid and the corresponding user.isActive == true. event is non-null. channels is non-empty. Notification preferences exist for user.
Postcondition	Notification delivered within 3 seconds via each specified channel (WebSocket and/or email/SMS). Notification persisted in notifications table with sent_at timestamp and message content. If delivery fails, notification inserted into retryQueue with retryCount = 0; up to 3 retries are scheduled. On 4th failure, entry moved to dead_letter_queue and admin alert raised.
Exception	Delivery failure does NOT throw — it is logged silently in deliveryFailureLog and queued for retry to prevent blocking lifecycle operations (INV-9, INV-10).

4.6 User.authenticate(pwd)

User.authenticate(pwd:String):Boolean	
Precondition	pwd is non-null and non-empty. this.isActive == true and this.isDeleted == false. this.failedLoginCount < 5 (account not locked out).
Postcondition	If hash matches: returns true; this.lastLoginAt = now(); this.failedLoginCount = 0. If hash does not match: returns false; this.failedLoginCount++; if count reaches 5, account is locked for 15 minutes and Admin is notified.

Exception

AccountLockedException if this.failedLoginCount >= 5 at time of call.

4.7 InspectionReport.attachPhoto(url)

InspectionReport.attachPhoto(url:URL):void**Precondition**

url is non-null and syntactically valid (HTTP/HTTPS scheme).
this.photoEvidence.size() < 5 (maximum 5 photos per inspection report per UC-13).

Postcondition

url is appended to this.photoEvidence. The report is persisted transactionally.

Exception

IllegalArgumentException if url is null or malformed. IllegalStateException if photo limit of 5 is reached.

5 Class and Method Invariants

Invariants are conditions that must hold for every instance of a class before and after every public operation. They are enforced by a combination of constructor guards, service-layer validation, database constraints, and unit-test assertions targeting $\geq 70\%$ coverage.

5.1 FoodDonation — Class Invariants

ID	Invariant
INV-1	<code>lifecycleLog.size() ≥ 1</code> once <code>submit()</code> has been called; at least one entry always exists after first transition.
INV-2	The sequence of <code>toState</code> values in <code>lifecycleLog</code> must form a valid path in the FSM — no gaps and no reversals are permitted.
INV-3	<code>status == lifecycleLog.last().toState</code> at all times; the <code>status</code> field is always consistent with the log (no divergence possible).
INV-4	<code>expiryDate > submittedAt.toDate()</code> ; a donation with already-expired food may not be submitted.
INV-5	<code>quantity > 0</code> ; zero or negative quantity is structurally invalid and rejected at construction time.
INV-6	<code>version ≥ 0</code> ; the optimistic lock counter is non-negative and monotonically increasing.

5.2 InspectionReport — Class Invariants

ID	Invariant
INV-7	One <code>InspectionReport</code> per <code>FoodDonation</code> (1:1); enforced by unique constraint on <code>donation_id</code> in DB and a pre-save check in <code>InspectionReportRepository</code> .
INV-8	<code>inspectedAt</code> is always set when <code>outcome</code> is not null; it is impossible to have a result without a timestamp.
INV-9	If <code>outcome == UNSAFE</code> then <code>hazardLevel</code> is not null and <code>violationType.length() > 0</code> .

5.3 NotificationService — Method Invariants (dispatch)

ID	Invariant
INV-10	Every call to <code>dispatch()</code> results in exactly one entry in the notifications table — never zero, never duplicate.
INV-11	<code>retryCount</code> never exceeds 3; on the 4th failure the notification is moved to <code>dead_letter_queue</code> and an Admin alert is raised.

5.4 User — Class Invariants

ID	Invariant
----	-----------

INV-12	passwordHash is never stored as plain text; it must match the regex pattern of a salted SHA-256 hash.
INV-13	role is immutable after creation; only Admin.changeRole() may update it, which mandatorily creates an audit log entry.
INV-14	failedLoginCount resets to 0 on every successful login.
INV-15	failedLoginCount never exceeds 5 without triggering a 15-minute account lockout.
INV-16	email is unique across all non-deleted user records (enforced by unique index).

5.5 LifecycleEntry — Immutability Invariant

ID	Invariant
INV-17	All fields of LifecycleEntry (fromState, toState, actorId, timestamp, notes) are set in the constructor and are thereafter final / read-only; no setter methods exist.
INV-18	The corresponding database row is INSERT-only; no UPDATE or DELETE permissions are granted to the application role at the DB level.
INV-19	timestamp is always set to now() at the moment of the transition call; it may never be backdated.

5.6 DonationMatchingEngine — Strategy Invariant

ID	Invariant
INV-20	algorithm is never null; a default strategy (ProximityMatchingStrategy) is set in the constructor and may only be replaced with a non-null value via setStrategy().
INV-21	match() always returns a List (possibly empty); it never returns null.
INV-22	The returned list is ordered by the active strategy's ranking criterion; ties are broken by expiry date ascending (most urgent first).

5.7 PickupTask — Class Invariants

ID	Invariant
INV-23	volunteerRef is null if and only if status == PENDING.
INV-24	acceptedAt is non-null if and only if status ∈ {ASSIGNED, EN_ROUTE, ARRIVED, PICKED_UP}.
INV-25	At most one active PickupTask per FoodDonation (unique constraint on donation_id where status != 'CANCELLED').

5.8 NGO — Class Invariants

ID	Invariant
----	-----------

INV-26	An NGO may only accept donations when verificationStatus == VERIFIED; any attempt by an unverified NGO raises a SecurityException.
INV-27	serviceArea is non-empty at the time of first donation acceptance; the matching engine must have at least one location polygon to evaluate proximity.

References

1. Gamma, E., Helm, R., Johnson, R., & Vlissides, J. Design Patterns: Elements of Reusable Object-Oriented Software. Addison-Wesley, 1994.
2. Bruegge, B. & Dutoit, A. Object-Oriented Software Engineering Using UML, Patterns, and Java. Prentice Hall.
3. Evans, E. Domain-Driven Design: Tackling Complexity in the Heart of Software. Addison-Wesley, 2003.
4. IEEE/ISO/IEC 29148:2018 — Systems and Software Engineering: Requirements Engineering.
5. FSSAI Guidelines on Food Handling and Safety Standards, Food Safety and Standards Authority of India.
6. Original FDLMS SRS — SE Lab 3B Group C, March 2026.
7. FDLMS SDD Part A — SE Lab Experiment 8A, April 2026.
8. VJTI SE Laboratory — Experiment 4A and 4B Use-Case Realisations, February–March 2026.